

Xolvon Backend — Handover Documentation

Official handover entry point for the Xolvon backend (NestJS 12, strict ESM, Node.js 22+). This folder is the canonical, code-verified reference. The root ``README.md`` and ``ARCHITECTURE.md`` predate some hardening work; where they disagree with this folder, the documents here (and the code) win.

Document index

Document	Purpose
<code>[`README.md`](README.md)</code> (this file)	From-zero setup, environment variables, commands, migrations, PDF build
<code>[`architecture.md`](architecture.md)</code>	Module map, data layer (SQLite/D1), storage port, auth model, full data model
<code>[`security.md`](security.md)</code>	Security headers, input sanitization, consolidated controls, audit results, limitations
<code>[`deployment-runbook.md`](deployment-runbook.md)</code>	Staging → production deployment, migration apply/rollback, admin bootstrap, monitoring status
<code>[`api-contract.md`](api-contract.md)</code>	Canonical endpoint contract (auth/role per route)
<code>[`decision-log.md`](decision-log.md)</code>	Locked decisions (DL-nnn), conflicts, open items
<code>[`production-operations.md`](production-operations.md)</code>	Environment separation, admin bootstrap, rollback, observability boundary
<code>[`qa-results.md`](qa-results.md)</code>	Live-deployment verification status (currently BLOCKED — recorded honestly)

1. Prerequisites

- `**Node.js**` v22.0.0+ (NestJS 12 runs native ESM)
- `**npm**` v10+
- No Cloudflare account, database, or bucket is needed for local development.

2. From-zero local setup

2.1 Clone and install

```
```bash
git clone <repository-url>
cd Project-SaaS-Xolvon
npm install
```
```

``.npmrc`` sets ``legacy-peer-deps=true`` (required by the ``.nestjs/throttler`` peer range, see DL-016); a clean ``npm install`` on a fresh clone must exit 0.

2.2 Create ``.env``

```
```bash
cp .env.example .env
```
```

``.env`` is gitignored — never commit it. The local defaults in ``.env.example`` run fully offline: ``APP_ENV=local``, ``DB_DRIVER=sqlite``, ``STORAGE_DRIVER=local-test``. You must supply only two values locally (names below; put real values in ``.env``, never in docs or Git):

- ``JWT_SECRET`` — any high-entropy string of at least 32 characters (e.g. generate one with ``openssl rand -hex 32``).
- ``ADMIN_BOOTSTRAP_EMAIL`` / ``ADMIN_BOOTSTRAP_PASSWORD`` — only needed when you run ``npm run seed:admin`` (section 5); not read by the HTTP server.

All other blank variables (``CLOUDFLARE_*``, ``R2_*``) are validated only when the matching driver is selected (``DB_DRIVER=d1`` / ``STORAGE_DRIVER=r2``) — see ``src/config/env.validation.ts``. The validator fails fast at bootstrap and never echoes values.

2.3 Build

```
```bash
npm run build
```
```

2.4 Apply database migrations (local SQLite)

> **Important:** the HTTP server does **not** auto-apply migrations at startup. ``src/main.ts`` never runs them; ``local.db`` starts empty. You must apply migrations once before the first run.

```
```bash
Applies all pending files in src/database/migrations/ to ./local.db
npx ts-node src/database/migrate.ts
```
```

(``npm run seed:admin`` also applies pending migrations first, so seeding implicitly migrates.)

2.5 Run locally

```
```bash
npm run start:dev # watch mode
or
npm run start:prod # node dist/main (after npm run build)
```
```

The API listens on ``http://localhost:PORT/api`` (default ``PORT=3000``). Smoke test – the health route is ``GET /api`` (it returns ``Hello World!`` and is exempt from all throttling groups):

```
```bash
curl -i http://localhost:3000/api
curl -s http://localhost:3000/api/courses # public catalog, paginated
```
```

3. Environment variable reference

Names and purposes only – **no** secret values belong in this repo, in Git, in logs, or in any document. The three templates are ``env.example`` (local, 16 names), ``env.staging.example`` and ``env.production.example`` (14 names each). Two further names are read at runtime but are optional (see rows marked `*runtime-optional*`).

| Variable | Purpose | Required |
|--|--|----------|
| <code>`APP_ENV`</code> | Runtime mode: <code>`local`</code> \ <code>`staging`</code> \ <code>`production`</code> . <code>`production`</code> enables HSTS in security headers. Optional, defaults <code>`local`</code> | |
| <code>`PORT`</code> | HTTP listen port (integer 1–65535). Optional, defaults <code>`3000`</code> | |
| <code>`DB_DRIVER`</code> | Persistence driver: <code>`sqlite`</code> (better-sqlite3 on <code>`local.db`</code>) \ <code>`d1`</code> (Cloudflare D1 REST adapter). Optional, defaults <code>`sqlite`</code> | |
| <code>`CLOUDFLARE_ACCOUNT_ID`</code> | Cloudflare account id for the D1 REST endpoint. Required when <code>`DB_DRIVER=d1`</code> | |
| <code>`CLOUDFLARE_D1_DATABASE_ID`</code> | D1 database id the REST adapter queries. Required when <code>`DB_DRIVER=d1`</code> | |
| <code>`CLOUDFLARE_API_TOKEN`</code> | Bearer token used per-query against the D1 REST API (never stored on the service instance). Required when <code>`DB_DRIVER=d1`</code> | |
| <code>`STORAGE_DRIVER`</code> | Object storage driver: <code>`local-test`</code> (in-process stub, <code>`http://local-storage.test`</code> URLs) \ <code>`r2`</code> (Cloudflare R2 via S3 presigning). Optional, defaults <code>`local-test`</code> | |
| <code>`R2_ENDPOINT`</code> | S3-compatible R2 endpoint (must be absolute <code>`https:`</code> URL). Its origin also feeds CSP source lists. Required when <code>`STORAGE_DRIVER=r2`</code> | |
| <code>`R2_BUCKET`</code> | R2 bucket name. Required when <code>`STORAGE_DRIVER=r2`</code> | |
| <code>`R2_ACCESS_KEY_ID`</code> | R2 access key id. Required when <code>`STORAGE_DRIVER=r2`</code> | |
| <code>`R2_SECRET_ACCESS_KEY`</code> | R2 secret access key. Required when <code>`STORAGE_DRIVER=r2`</code> | |
| <code>`JWT_SECRET`</code> | HS secret for signing/verifying access tokens (<code>`AuthGuard`</code>). Always required (min 32 chars) | |
| <code>`FRONTEND_URL`</code> | Primary allowed browser origin; CORS and request-integrity fallback when <code>`CORS_ALLOWED_ORIGINS`</code> is unset. Always required (valid <code>`http(s)`</code> URL) | |
| <code>`CORS_ALLOWED_ORIGINS`</code> | Comma-separated explicit whitelist of allowed origins (credentials mode; wildcards rejected). Also the request-integrity guard's allowed-origin set. Falls back to <code>`FRONTEND_URL`</code> , then <code>`http://localhost:3001`</code> . Runtime-optional (not in the example files) | |
| <code>`CSP_CONNECT_SRC`</code> | Comma/space-separated origins added to CSP <code>`connect-src`</code> / <code>`style-src`</code> / <code>`img-src`</code> / <code>`font-</code> | |

src`. Malformed entries are dropped. Defaults to `self`. | Runtime-optional (not in the example files) |
 | `TRUST\_PROXY\_HOPS` | Express `trust proxy` hop count for client-IP resolution behind the edge proxy.
 Unset/blank → `false` (fail closed: `X-Forwarded-For` cannot mint fresh throttle buckets); non-numeric →
 bootstrap aborts. Staging/production templates set `1`. | Optional (validated at bootstrap, not by
 `env.validation.ts`) |
 | `ADMIN\_BOOTSTRAP\_EMAIL` | Email of the first admin account, consumed only by `npm run seed:admin`. |
 Required for `seed:admin` |
 | `ADMIN\_BOOTSTRAP\_PASSWORD` | Password for the first admin account, consumed only by `npm run seed:admin`.
 Required for `seed:admin` |

Cross-field validation lives in `src/config/env.validation.ts` and keys off
 `DB\_DRIVER`/`STORAGE\_DRIVER` – not off `APP\_ENV`.

4. Commands

| Command | What it does |
|----------------------|---|
| --- --- | |
| `npm run test:esm` | Unit/behavior suite (Jest with `--experimental-vm-modules`, via cross-env). Current baseline on this branch: 42 suites / 399 tests. |
| `npm run test:e2e` | End-to-end suite (`test/jest-e2e.json` harness with overridden providers). Baseline: 107 tests. Runs fully against in-memory/local SQLite – zero cloud credentials. |
| `npm run build` | `nest build` → `dist/`. |
| `npm run lint` | `oxlint src/ test/`. |
| `npm run format` | Prettier over `src/` and `test/`. |
| `npm run test:cov` | Coverage report. |
| `npm run seed:admin` | Controlled first-admin bootstrap (section 5). |
| `npm run docs:pdf` | Regenerate committed PDFs (section 6). |

Plain `npm test` (`jest` without the ESM flag) is **not** usable with this
 ESM-only codebase – always use `test:esm` / `test:e2e` (DL-008).

5. Adding and running a migration

Migrations are plain SQL files in `src/database/migrations/`, applied by
 `src/database/migrate.ts` in **filename sort order**, tracked in the
 `\_migrations` table (filename + SHA-256 content hash). Rules:

- Add** a new file `NNNN\_short\_description.sql` continuing the numeric prefix (next free number after `0007\_...`). Name only – content of applied files is immutable.
- Never edit an applied migration.** The runner compares the recorded hash and throws on drift (`"changed after it was applied"`), fail-closed. Fix forward with a new file.
- Statements are split with a string/comment/trigger-aware splitter, so plain `;-`-separated DDL is safe.
- Run locally:** `npx ts-node src/database/migrate.ts` (applies to `./local.db` under the current working directory; prints `applied/skipped` counts). The HTTP server itself never runs it.
- Run against D1 (staging/production):**
 `npx wrangler d1 migrations apply xolvon-staging --remote` – the database name and `migrations\_dir` are configured in `wrangler.jsonc`. See deployment-runbook.md for the full procedure and rollback policy (forward-only; compensating migration + deployment revert).
- The `src/database/migrations/\_reference/` folder is a reference copy and is **not** applied (the runner only enumerates top-level `\*.sql` files).
- Schema changes must be preceded by a `decision-log.md` entry (RULES §84);
 `SCHEMA.md` remains the schema source of truth.

6. Generating documentation PDFs

Per DL-019, official PDFs are produced with Playwright/Chromium and committed:

```
```bash
npm run docs:pdf
```
```

- Runs `scripts/docs-pdf.mjs`: renders every `docs/\*.md` to `docs/pdf/<name>.pdf` (A4, Chromium print pipeline).
- Requires the Chromium browser binary once per machine:
 `npx playwright install chromium`.
- Output is deterministic for the same Markdown input; regenerate and

secret-scan whenever these docs change.

- `docs/Xolvon-API-Documentation.pdf` (and its generator `scripts/generate-api-pdf.py`) is **not** part of this workflow – it came from an out-of-band commit, is non-conforming to DL-019, and is scheduled for removal/replacement at plan T9. See DL-026 in [`decision-log.md`](decision-log.md).

7. House rules for contributors

- No secret or credential values in docs, tests, logs, Postman files, PDFs, or Git history – variable **names** only (DL-024, cleanup policy DL-025).
- No production deployment claims without real target + HTTPS domain + verification evidence (current status: blocked, see [`qa-results.md`](qa-results.md)).
- Endpoint or auth-model changes require updating [`api-contract.md`](api-contract.md) and a `decision-log.md` entry in the same change.

Code-verified architecture reference for the NestJS 12 (strict ESM, Express platform) backend. Every claim below cites the file that implements it.
Companion documents: [`api-contract.md`](api-contract.md) (endpoint contract), [`security.md`](security.md) (controls), [`deployment-runbook.md`](deployment-runbook.md) (operations), [`decision-log.md`](decision-log.md) (why).

1. Request lifecycle

Bootstrap order in `src/main.ts` (all guards/pipes below are process-wide):

...

Request → Express (trust proxy: `TRUST_PROXY_HOPS`, `src/main.ts:25-28`)
→ Helmet security headers (`enableSecurityHeaders`, `src/main.ts:35-39`)
→ CORS (explicit origin whitelist + credentials, `src/main.ts:43-50`)
→ Global prefix `/api` (`src/main.ts:17`)
→ APP_GUARD #1 `ThrottlerGuard` (`src/app.module.ts:65`)
→ APP_GUARD #2 `RequestIntegrityGuard` (`src/app.module.ts:66`)
→ `@UseGuards(AuthGuard, RolesGuard)` where declared per route/controller
→ `ValidationPipe { whitelist, forbidNonWhitelisted, transform }` (`src/main.ts:53-59`)
→ Controller → Service → `DatabaseService` token / `StoragePort` token
→ `AllExceptionsFilter` (safe JSON error envelope, `src/main.ts:62`)

...

There is no middleware layer and no `NestModule` configuration anywhere – the only `app.use()` is `Helmet` inside `src/security/security-headers.ts`.
`AuthGuard` is deliberately **not** global: routes that don't attach it are public (`catalog/search/home/collective/marketplace` GETs); `@Public()` is metadata consumed by `AuthGuard` only where it is attached (`src/auth/guards/auth.guard.ts:23-30`).

2. NestJS module map

`src/app.module.ts:30-61` (import order):

| Module | Directory | Responsibility |
|---|--|--|
| <code>ConfigModule</code> | (global) | <code>src/config/</code> <code>.env</code> load + fail-fast <code>validateEnvironment</code> (<code>env.validation.ts</code>); CORS, CSP, trust-proxy, throttle resolvers |
| <code>ThrottlerModule</code> | (global, via <code>APP_GUARD</code>) | <code>src/config/throttle.config.ts</code> 5 named limit groups (\$5.3) |
| <code>DatabaseModule</code> | <code>src/database/</code> | <code>DatabaseService</code> DI token; driver factory <code>sqlite</code> ↔ <code>DI REST adapter</code> ; migration runner |
| <code>AuthModule</code> | <code>src/auth/</code> | Register/login/refresh/logout, Argon2id hashing, JWT issue/verify, hashed refresh sessions, <code>AuthGuard</code> / <code>RolesGuard</code> |
| <code>EnrollmentsModule</code> | <code>src/enrollments/</code> | Entitlement checks, idempotent activation, admin revoke |
| <code>OrdersModule</code> | <code>src/orders/</code> | Server-computed order totals, payment-proof submission (anti-IDOR), verify/activate/cancel transitions |
| <code>CoursesModule</code> | <code>src/courses/</code> | Published-only catalog, draft-gated admin CRUD, publish gates, slug uniqueness |
| <code>HomeModule</code> | <code>src/home/</code> | Published-only aggregation (featured courses/projects/marketplace/collective), no N+1 |
| <code>MarketplaceModule</code> | <code>src/marketplace/</code> | Showcase-only SaaS catalog; <code>externalUrl</code> minimal https validation (DL-013) |
| <code>AuditModule</code> | <code>src/audit/</code> | Internal <code>record()</code> writer for <code>admin_audit_logs</code> ; no public endpoint, metadata never exits the API |
| <code>CollectiveModule</code> | <code>src/collective/</code> | Talent roster (draft/published), never exposes email/phone on public reads |
| <code>ProjectsModule</code> + <code>ProjectMediaModule</code> + <code>ProjectMembersModule</code> | <code>src/projects/</code> , <code>src/project-media/</code> , <code>src/project-members/</code> | Portfolio CRUD, media attach/detach (private object keys never public), member assignment (<code>type</code> / <code>mediaType</code> / <code>role</code> are controlled strings, DL-014) |
| <code>LessonsModule</code> | <code>src/lessons/</code> | Lesson CRUD/reorder/publish; <code>GET /api/lessons/:id/video-url</code> mints 300s signed URLs gated by active enrollment |
| <code>CourseResourcesModule</code> | <code>src/course-resources/</code> | Resource attach (<code>pdf</code> <code>resource</code> <code>assignment</code>) under lessons |
| <code>AdminModule</code> | <code>src/admin/</code> | Read-only admin surfaces: users, orders, overview counters |
| <code>MediaModule</code> | <code>src/media/</code> | <code>STORAGE_PORT</code> binding + admin upload-url/confirm/read-url endpoints |
| <code>ProgressModule</code> | <code>src/progress/</code> | Lesson completion state, ownership from JWT subject |
| <code>SearchModule</code> | <code>src/search/</code> | Cross-entity FTS5 search (courses/projects/marketplace), published-only |

Health endpoint: ``GET /api`` (``src/app.controller.ts``) – returns a static string, exempt from every throttle group via ``@SkipThrottle(HEALTH_SKIP_THROTTLE)``.

3. Data layer – one token, two drivers

All services inject the `**`DatabaseService`` class as DI token`**`; the factory in ``src/database/database.module.ts:25-44`` selects the implementation from ``DB_DRIVER``:

3.1 ``sqlite`` (local development & e2e)

``DatabaseService`` (``src/database/database.service.ts``) opens ``better-sqlite3`` on ``../local.db`` (relative to cwd) with ``foreign_keys = ON``, exposing ``queryAll``, ``queryOne``, ``execute`` – always positional ``?`` bind parameters (prepared statements are mandatory per HANDBOOK_BACKEND §4.1). Zero cloud credentials needed.

3.2 ``dl`` (staging/production)

``DlRestAdapter`` (``src/database/dl-rest.adapter.ts``) wraps ``DlService`` (``src/database/dl.service.ts``) behind the same three methods. ``DlService`` POSTs ``{sql, params}`` to ``https://api.cloudflare.com/client/v4/accounts/<CLOUDFLARE_ACCOUNT_ID>/dl/database/<CLOUDFLARE_DL_DATABASE_ID>/query`` with ``Authorization: Bearer <CLOUDFLARE_API_TOKEN>`` `**resolved on demand per query and never stored as an instance field**` (hardening invariant, ``dl.service.ts:77-79``). Timeout 10s; timeouts→504, network/non-OK→502 – the adapter never leaks the upstream body. ``onModuleInit`` fail-fasts on missing ``CLOUDFLARE_*`` values. (``src/database/dl.module.ts`` exists but is imported nowhere – dead code kept for reference.)

Migrations run through the same ``execute()`` contract, so the runner (``src/database/migrate.ts``) works on either driver; the operational path for DL is ``wrangler dl migrations apply`` (see runbook). Applied files are recorded in ``_migrations`` with a SHA-256 hash; hash drift throws (fail-closed, forward-only policy).

4. Storage layer – ``StoragePort`` interface-first (DL-007)

``src/media/storage.port.ts`` defines the token ``STORAGE_PORT`` and the interface:

```
````ts
createUploadUrl(request: { key; contentType; contentLength }): Promise<string>
createReadUrl(key: string, expiresIn?: number): Promise<string>
confirmUpload(key: string): Promise<void>
````
```

``src/media/media.module.ts:12`` binds the implementation from ``STORAGE_DRIVER``:

| Driver | Class | Behavior |
|---------------------------|--|---|
| <code>`local-test`</code> | <code>`src/media/local-test-storage.service.ts`</code> | Deterministic stub URLs (<code>`http://local-storage.test/...`</code>); <code>`confirmUpload`</code> records keys in memory. Used by all unit/e2e runs – tests never touch the network. |
| <code>`r2`</code> | <code>`src/media/r2-storage.service.ts`</code> | S3 SDK v3 against R2 (<code>`region: 'auto'`</code>): presigned <code>**PUT 3600s**</code> (signable headers: content-type/content-length) and presigned <code>**GET default 300s**</code> . Credentials only from env, never in responses. |

Consumers (``src/media/media.service.ts``, ``src/lessons/lessons.service.ts``) inject the token and are unchanged when drivers are swapped. Private object keys never appear in public API responses; entitlement is checked `**before**` a read URL is minted (server owns trust).

5. Auth model – JWT Bearer + hashed refresh sessions

5.1 Tokens

– `**Access:**` JWT signed/verified with ``JWT_SECRET``, TTL `**15m**` (``src/auth/auth.service.ts:132-133, 211-212``). Sent as

```
`Authorization: Bearer <token>`; extraction and verification in
`src/auth/guards/auth.guard.ts` (generic 401 messages, no token echo).
- **Refresh** cryptographically random string returned once at login; the
  database stores only its **SHA-256 hash** in `sessions.refresh_token`
  (`auth.service.ts:31-33, 136-149`) alongside `user_id`, `ip_address`,
  `user_agent`, `expires_at`. `refresh` looks the session up by hash and
  purges expired rows; `logout` deletes the session by hash
  (`auth.service.ts:173-225`). Plaintext refresh tokens can never be read back
  from a leaked database.
```

5.2 CSRF model (explicit non-decision to adopt cookies)

The cookie-session CSRF pattern (synchronizer token in a cookie) was ****NOT adopted****. The API is Bearer-only – no authentication happens via cookies – so classic cross-site request forgery cannot carry credentials automatically. Instead, browser state-changing requests are constrained by the ****request-integrity guard****: `POST`/`PATCH`/`DELETE` requests must not declare `Sec-Fetch-Site: cross-site`, and any `Origin`/`Referer` they carry must be in the explicit allowed-origin set (`CORS\_ALLOWED\_ORIGINS`, falling back to `FRONTEND\_URL`). Non-mutating requests are unaffected, and headerless service-to-service clients remain compatible; authentication is still required independently on every protected route. This is the locked decision ****DL-018**** (with DL-003 keeping the repo's JWT+refresh model over the handbook's `getSession()` sample); implementation
`src/common/guards/request-integrity.guard.ts`, rationale
[`security.md`](security.md#consolidated-security-controls).

5.3 Rate limiting

`@nestjs/throttler` with five named groups, all 60s windows
(`src/config/throttle.config.ts`): `default` 100/min (undecorated routes),
`search` 30/min (`GET /api/search`), `catalog` 30/min (courses/projects/
marketplace/collective listings + `GET /api/home`), `signed` 10/min (lesson
video URL, media upload-url/read-url), `order` 10/min (`POST /api/orders`).
`POST /api/auth/login` and `POST /api/auth/register` are tightened inline to
****5/min per IP**** (DL-012/DL-016). Client-IP accuracy behind an edge proxy is
controlled by `TRUST\_PROXY\_HOPS`; locally it fails ****closed**** to `false` so
`X-Forwarded-For` cannot mint fresh buckets (`src/config/trust-proxy.ts`).

5.4 Authorization

`RolesGuard` enforces `@Roles('admin')` on all admin surfaces; roles come
exclusively from the `users` row / verified JWT – never from request input
(DL-005). First-admin creation is a controlled seed command
(`npm run seed:admin`), not a public route.

6. Data model – 19 tables + FTS5

Schema source of truth: `SCHEMA.md`; applied set:
`src/database/migrations/0000...0007` (19 regular tables + 3 FTS5 virtual
tables; plus the runtime bookkeeping table `\_migrations`). Conventions
(HANDBOOK_BACKEND §3.1): TEXT UUID v4 primary keys, ISO 8601 TEXT timestamps,
INTEGER money (Rupiah) and booleans (0/1), TEXT + `CHECK` for enums,
prepared statements only.

6.1 Tables by domain

| Domain | Tables | Notes |
|-------------|---|--|
| Identity | `users`, `sessions` | `users.role` CHECK `user\ admin`; `users.status` defaults `active` (no CHECK constraint); `sessions.refresh_token` stores SHA-256 hash (§5.1) |
| Course | `courses`, `lessons`, `course_resources`, `course_tags` | `courses.slug` UNIQUE; `lessons.order_index` sequencing, `is_preview` 0/1, `video_object_key` private; resource `type` `pdf\ resource\ assignment` |
| Transaction | `orders`, `order_items`, `payment_proofs` | `orders.status` **final** `pending\ paid\ cancelled` (DL-002); server-computed `orders.amount` (`CHECK > 0`); proofs store R2 object keys + `uploaded_by`, anti-IDOR at submit (DL-006) |
| Entitlement | `enrollments` | UNIQUE `(user_id, course_id)` prevents double enrollment; status **final** `active\ revoked` (DL-004); idempotent upsert on activation |
| Learning | `progress` | Per-user lesson completion; ownership from JWT subject |
| Portfolio | `projects`, `project_media`, `project_tags` | Narrative fields Problem→Solution→Tech→Result; |

```

`project_media.object_key` private, `media_type` controlled string (DL-014) |
| Collective | `collective_members`, `project_members` | `project_members(project_id, member_id)` is the
Portfolio+Collective M:N bridge with explicit `role`; `display_order` for listings |
| Marketplace | `marketplace_items`, `marketplace_media` | Showcase only – no transactions (SCHEMA.md §60);
`external_url` https-minimal (DL-013) |
| Security | `admin_audit_logs` | Append-only trail for admin state mutations
(`verify/activate/cancel/revoke/publish/create/update/delete`); never exposed publicly |
| Search | `course_fts`, `project_fts`, `marketplace_fts` | FTS5 virtual tables kept in sync by 9
insert/delete/update triggers (`0007_fts.sql`); power `GET /api/search` |

```

6.2 Relationships

```

```text
users 1-N sessions (user_id FK)
users 1-N orders (user_id FK)
users 1-N enrollments N-1 courses (both FKs; UNIQUE pair)
users 1-N progress (user_id FK; lesson-scoped rows)
courses 1-N lessons (course_id FK, ON DELETE CASCADE)
courses 1-N course_resources (via lesson/course, course_resources FK)
courses N-M course_tags
orders 1-N order_items N-1 courses (order_id, course_id FKs)
orders 1-N payment_proofs (order_id FK; object keys only)
projects 1-N project_media (project_id FK)
projects N-M collective_members (via project_members: composite PK
 (project_id→projects FK CASCADE, member_id – lookup enforced in code))
projects N-M project_tags
marketplace_items 1-N marketplace_media
users 1-N admin_audit_logs (actor_user_id FK)
```

```

Critical invariants (HANDBOOK §3.3, DL-002/DL-004): entitlement is
`enrollments.status='active'` per `(user, course)`; `expires\_at` columns are
enforced in queries; no delete/role-change endpoints exist in V1, so
soft-state transitions (revoke/cancel/unpublish) are the only mutations, and
each writes an audit row.

7. Framework notes

- **ESM-only**: NestJS 12 with `"type": "module"`; hence
`NODE\_OPTIONS=--experimental-vm-modules` in test scripts (DL-008) and
`.js`-extension imports in source.
- **Deviation from HANDBOOK_BACKEND §8** (Langkah 6/8): the handbook's
`vinext`/Workers binding path targets the frontend/Workers deployment. The
backend is Node/Express and reaches D1 through the REST adapter behind the
`DatabaseService` token – a documented deviation, conflict K-06 in
[`decision-log.md`](decision-log.md).
- **Error surface**: `AllExceptionsFilter` returns
`{statusCode, message, error, timestamp, path}` with a generic 500 message –
no stack traces, upstream bodies, or secrets.

Security

Official security documentation for the XOLVON backend. Decisions behind the controls listed here live in [`decision-log.md`](decision-log.md); endpoint and auth contracts live in [`api-contract.md`](api-contract.md).

Security headers

All responses – including CORS preflight replies and error bodies – carry a strict header set produced by [Helmet](https://helmetjs.github.io/), wired globally in `src/main.ts` before CORS, the validation pipe, and the exception filter. The configuration lives in `src/security/security-headers.ts` and is covered by `src/security/security-headers.spec.ts`.

| Header | Value | Rationale |
|--|--|--|
| <code>Content-Security-Policy</code> | <code>`default-src 'none'; script-src 'self'; frame-ancestors 'none'; connect-src/style-src/img-src/font-src 'self' + configured origins`</code> | Deny-by-default policy for any embedded resource. The API serves JSON, so nothing legitimate needs inline scripts or framing; <code>unsafe-inline</code> and <code>unsafe-eval</code> are never emitted (DL-021). <code>frame-ancestors 'none'</code> blocks clickjacking of every response. |
| <code>Strict-Transport-Security</code> | <code>`max-age=31536000; includeSubDomains`</code> | – only when <code>APP_ENV=production</code>
One-year HTTPS pinning in production. Local and staging intentionally run over plain HTTP, so the header is absent there to avoid locking browsers out of dev origins. |
| <code>X-Frame-Options</code> | <code>DENY</code> | Legacy framing protection alongside CSP <code>frame-ancestors</code> , for older clients. |
| <code>X-Content-Type-Options</code> | <code>nosniff</code> | Stops browsers from MIME-sniffing responses into executable/script contexts. |
| <code>Referrer-Policy</code> | <code>strict-origin-when-cross-origin</code> | Cross-origin requests only receive the origin as referrer, so tokens or identifiers in URLs never leak to third parties; HTTPS→HTTP downgrades send nothing. |
| <code>X-Powered-By</code> | removed | Express' default fingerprint is disabled at the app level and stripped by Helmet, reducing reconnaissance surface. |

CSP origin sources

The fetch/media source lists (`connect-src`, `style-src`, `img-src`, `font-src`) are built at bootstrap from environment variables, always prefixed with `'self'` as the safe default when nothing is configured:

| Variable | Feeds | Notes |
|------------------------------|-----------------------|---|
| <code>CSP_CONNECT_SRC</code> | all four source lists | Comma- or space-separated absolute <code>http(s)</code> URLs. Each entry is normalized to its origin; malformed or non- <code>http(s)</code> values are dropped and never echoed into the header. |
| <code>R2_ENDPOINT</code> | all four source lists | Only the URL origin is used (e.g. <code>https://<account-hash>.r2.cloudflarestorage.com</code>); bucket names, paths, and credentials are excluded. |

Approved baseline example (staging/production final origins remain OPEN per DL-021 until the owner confirms domains):

```
```text
CSP_CONNECT_SRC=https://app.xolvon.com,https://admin.xolvon.com
```
```

No secrets, tokens, or credentials appear in any header value.

Verification

- Unit/config level: `src/security/security-headers.spec.ts` (runs under `npm run test:esm`) asserts the full header set on a real Nest testing app for both `production` (HSTS present) and non-production (HSTS absent), plus origin-parsing edge cases and the no-`unsafe-*` rule.
- Surface level: `curl -s -i` against the local server with and without `APP_ENV=production` (evidence in the plan notepad, `.omo/.../evidence/`).

Input sanitization

Free-text input is normalized by a bounded, deterministic policy before it can be persisted, implemented in `src/common/sanitization/` and wired per field through the existing global `ValidationPipe` (`whitelist/forbidNonWhitelisted/transform` in `src/main.ts` are unchanged).

****Policy – `sanitizeText(value, { maxLength, allowHtml })`:****

- Content fields intended for rendered HTML (`allowHtml: false`, the default and the only mode in use) have every tag-shaped substring (`<...>`) removed (a fixed-point sweep re-checks defensively), so payloads such as `- Invisible control characters (NUL, ESC, and the rest of C0/DEL except `\t` `\n` `\r`) are stripped.
- Everything else – including ordinary Unicode (emoji, accented Latin, CJK), ampersands, and quotes – is preserved byte-for-byte. No trimming, case changes, or truncation: a valid request's visible content is never altered.
- Oversized values are rejected with `400`, not truncated: every sanitized field carries an explicit `@MaxLength`, and the helper's `maxLength` option throws (never echoes the value) for programmatic callers.

****Policy – JSON metadata (`parseSafeJson` / `@IsSafeMetadata`):****

- Metadata TEXT fields that may carry structured content (project `tech\_stack`) follow "JSON-shaped means JSON-valid": a value starting with `[` or `{` must parse as a complete JSON document or the request is rejected with `400`; malformed payloads are never stored for read mappers to silently degrade. Legacy comma-separated lists remain valid, matching `parseTechStack` in `src/projects/mappers/project.mapper.ts`.

****Sanitized fields (create + update DTOs):**** `courses.description`, `lessons.content`, `projects.summary/problem/solution/result`, `marketplace.description`, `collective.bio`, `orders.notes` (all via `@SanitizedText`), plus `projects.techStack` via `@IsSafeMetadata`. Plain-text and identity fields (titles, slugs, names, roles, object keys, tokens) are deliberately not rewritten – their length/type bounds stay as before.

****Known limits:**** the policy neutralizes markup at the write boundary; renderers must still apply their own contextual escaping (defense in depth, on top of the CSP in [Security headers](#security-headers)). String items inside arrays (`skills`, `capabilities`) keep their existing type/size bounds and are out of scope for tag stripping.

Verification

- Unit level: `src/common/sanitization/sanitization.spec.ts` (runs under `npm run test:esm`) – 27 tests covering tag removal, reassembly payloads, Unicode preservation, control characters, length bounds, idempotence, malformed/valid JSON metadata, and the real content DTOs.
- Surface level: the T7 block in `test/app.e2e-spec.ts` asserts that markup is gone from the actual DB `INSERT` parameters, oversized content returns 400 before any write, and malformed `techStack` JSON returns 400; manual `curl` evidence lives in the plan notepad, [`.omo/notepads/xolvon-addendum-hardening-docs/evidence/`](#).

Consolidated security controls

Every control introduced or hardened by the addendum plan ([`.omo/plans/xolvon-addendum-hardening-docs.md`](#)), mapped to its code and to the addendum §5 checklist ([`.omo/drafts/xolvon-addendum-hardening-docs.md`](#), Scope ledger – In scope). All entries verified against the source tree on 2026-09-12.

| Control (plan task) | Code | Tests | Addendum §5 checklist item | Decision |
|---|------|-------|----------------------------|----------|
| --- --- --- --- --- | | | | |
| **Request integrity / CSRF posture** (T3) `src/common/guards/request-integrity.guard.ts` – global <code>APP_GUARD</code> (`src/app.module.ts:66`); <code>POST/PATCH/DELETE</code> only; rejects <code>Sec-Fetch-Site: cross-site</code> and any <code>Origin`/`Referer`</code> outside the allowed set; <code>GET/HEAD/OPTIONS</code> and headerless non-browser requests pass   <a href="#">`request-integrity.guard.spec.ts` + e2e + curl matrix "Bearer-only request-integrity/Origin/Fetch-Metadata guard for mutation routes and negative tests" – no cookie-session CSRF tokens DL-018 (with DL-003) | | | | |
| **Security headers** (T4) `src/security/security-headers.ts` wired at `src/main.ts:35-39` , before <code>CORS/pipes/filter</code> – see [Security headers](#security-headers) `src/security/security-headers.spec.ts` + curl headerslocal/ <code>APP_ENV=production`</code> "Helmet security headers with production-only HSTS and explicit CSP/frame/nosniff/referrer policy" DL-021 | | | | |

```
| **CORS whitelist** (T5) | `src/config/cors.ts` (`resolveCorsOrigins`) + `app.enableCors({ origin,
credentials: true })` at `src/main.ts:43-50`; wildcards rejected with credentials; local fallback
`http://localhost:3001` | `cors.spec.ts` + curl allowed/denied origins | "Explicit CORS origin whitelist
per environment" | DL-020 |
| **Route-specific throttling** (T6) | `src/config/throttle.config.ts` (5 named groups: default 100, search
30, catalog 30, signed 10, order 10 / min) + `@Throttle` overrides on controllers; auth login/register
5/min (`src/auth/auth.controller.ts:33,42`); health skips all groups (`src/app.controller.ts:10`) |
`throttle.config.spec.ts` + e2e burst tests per group | "Route-specific throttling for public
search/catalogs, signed URL, and order creation" | DL-012, DL-016 |
| **Trust proxy / client-IP** (T6) | `src/config/trust-proxy.ts` + `app.set('trust proxy', ...)`
(`src/main.ts:25-28`): unset → `false` (fail closed against `X-Forwarded-For` spoofing of throttle
buckets), `1` for one proxy hop, malformed → bootstrap aborts | `trust-proxy.spec.ts` + live burst with
rotating XFF | Throttling correctness prerequisite of the checklist item above | DL-016 (extension) |
| **Input sanitization** (T7) | `src/common/sanitization/` (`sanitizeText`, `parseSafeJson`,
`@SanitizedText`, `@IsSafeMetadata`) wired through the existing global `ValidationPipe` – see [Input
sanitization](#input-sanitization) | `sanitization.spec.ts` (27 tests) + e2e DB read-back | "Input/content
sanitization policy for HTML-bearing strings and length limits, with tests and no silent data corruption" |
RULES §95, DL-023 lineage |
| **Password hashing** | Argon2id via `PasswordService` (`src/auth/`); never exposed in responses | auth
unit/e2e | Baseline (pre-addendum), kept | DL-003 context |
| **Refresh-session hygiene** | SHA-256 hashed `sessions.refresh_token`, IP/user-agent capture, expiry
purge on refresh, delete on logout (`src/auth/auth.service.ts:136-225`) | auth unit/e2e | Baseline, kept |
DL-003 |
| **Input contract enforcement** | Global `ValidationPipe` `whitelist` + `forbidNonWhitelisted` +
`transform` (`src/main.ts:53-59`); DTOs reject unknown properties | every controller spec relies on it |
Baseline, kept | – |
| **Safe error surface** | `AllExceptionsFilter` (`src/common/filters/all-exceptions.filter.ts`): generic
500, no stack/upstream-body/secrets | filter + e2e assertions | Baseline, kept | – |
| **Immutable admin audit** | `AuditService.record()` → `admin_audit_logs` for
verify/activate/cancel/revoke/publish/create/update/delete; no public endpoint, metadata never leaves the
API | audit unit + admin e2e | Baseline, kept (HANDBOOK §7 obligation) | – |
```

Dependency audit status

`npm audit --json` was captured during plan T2 on ****2026-09-12****
(evidence: ``.omo/notepads/xolvon-addendum-hardening-docs/evidence/t2-audit.json``,
exit code 1 as expected with nonzero findings):

```
Severity	Count
Critical	0
High	4
Moderate	1
Low	2
**Total**	**7**
```

Remediation of the high/moderate residuals and the Dependabot-vs-scanner
policy are plan T13 work; no paid scanner has been adopted (DL-023 – OPEN).
No claim of a clean audit is made here.

Known limitations (honest state)

- **Production D1/R2 live path is BLOCKED, not deployed.**** There is no
verified staging/production deployment target, secret store, or HTTPS
domain in this workspace, and no live evidence is fabricated – see
[`qa-results.md`](qa-results.md). All security verification above is local
(unit/e2e/curl against locally booted instances with non-secret config).
- **CSP and CORS final origins are OPEN DECISIONS.**** Staging/production
frontend origins and `CSP\_CONNECT\_SRC` values await owner confirmation
(DL-020, DL-021, DL-027); until then the lists default to `self` plus
the configured `R2\_ENDPOINT` origin only.
- **Monitoring/alerting is interface-only.**** `MonitoringPort`
(`src/observability/monitoring.port.ts`) has no implementation or provider;
vendor and recipients remain OPEN (DL-022).
- **The request-integrity guard is a browser-CSRF control, not an auth
layer.**** A `POST/PATCH/DELETE` with no `Origin`, no `Referer`, and no
`Sec-Fetch-Site` header (curl, Postman, server-to-service clients) passes
the guard by design; such clients still need valid Bearer credentials on
every protected route. This is the explicit non-browser bypass policy
recorded under DL-018, not an oversight.
- **Sanitization neutralizes markup at the write boundary only****; renderers

must still apply contextual escaping (defense in depth over CSP).

Step-by-step operating procedure for promoting this backend to staging and production, including migrations, rollback, first-admin bootstrap, and the (currently open) monitoring decision. Read [``production-operations.md``](production-operations.md) for policy and [``qa-results.md``](qa-results.md) for the verified state.

0. Current honest status

- **Local**: fully working — SQLite (``DB_DRIVER=sqlite``) + local-test storage (``STORAGE_DRIVER=local-test``), no cloud credentials.
- **Staging/production**: **NOT** deployed and **NOT** verified. The D1/R2 live path is blocked for lack of a deployment target, secret stores, wrangler access, and final HTTPS domains (recorded in [``qa-results.md``](qa-results.md)). Nothing in this runbook may be marked done without real evidence; do not claim success from a local dry run.
- **Unblock requirements** (from ``qa-results.md``): separate staging/production secret stores with valid D1/R2/JWT values, an authenticated deployment provider/target for the Node service, pre-migration backups, log access, and real HTTPS domains for both environments.

1. Deployment architecture (documented deviation)

The backend runs as a **Node/Express NestJS service** (``@nestjs/platform-express``, ``node dist/main``). Cloudflare D1 is reached **over** its REST API through the ``D1RestAdapter`` behind the ``DatabaseService`` token (``DB_DRIVER=d1``), and R2 through the S3-compatible SDK behind ``STORAGE_PORT`` (``STORAGE_DRIVER=r2``).

HANDBOOK_BACKEND.md §8 Langkah 6/8 shows swapping ``DatabaseService`` to a Workers ``env.DB`` binding and deploying via ``vinext``/``@vinext/cloudflare`` — that path belongs to the **frontend/Workers deployment**. For this backend it is a **documented deviation** (conflict K-06 in [``decision-log.md``](decision-log.md)): the service is not a Workers runtime target; do not attempt to deploy ``dist/main`` with vinext.

2. Configuration per environment

Copy names only from the committed templates and populate values in the target's secret store — never in Git, docs, logs, or shell history:

```
Template	Fixed shape (non-secret)
`.env.staging.example`	`APP_ENV=staging`, `DB_DRIVER=d1`, `STORAGE_DRIVER=r2`, `TRUST_PROXY_HOPS=1`
`.env.production.example`	`APP_ENV=production`, `DB_DRIVER=d1`, `STORAGE_DRIVER=r2`,
`TRUST_PROXY_HOPS=1` |
```

Both templates still need real values for ``CLOUDFLARE_ACCOUNT_ID``, ``CLOUDFLARE_D1_DATABASE_ID``, ``CLOUDFLARE_API_TOKEN``, ``R2_ENDPOINT``, ``R2_BUCKET``, ``R2_ACCESS_KEY_ID``, ``R2_SECRET_ACCESS_KEY``, ``JWT_SECRET``, ``FRONTEND_URL``, and optionally ``PORT``. In addition, set the two runtime-optional names per environment once domains are decided (DL-020/DL-021/DL-027):

- ``CORS_ALLOWED_ORIGINS`` — final staging/production frontend origins (comma separated). **OPEN** — owner has not supplied final domains. Wildcards are rejected when credentials mode is on.
- ``CSP_CONNECT_SRC`` — frontend origins allowed in CSP fetch/source lists. **OPEN** until confirmed; unset is safe (defaults to ``'self'`` + ``R2_ENDPOINT`` origin).

Validation is fail-fast at bootstrap (``src/config/env.validation.ts``): ``d1`` requires the three ``CLOUDFLARE_*`` names; ``r2`` requires the four ``R2_*`` names; ``JWT_SECRET`` ≥ 32 chars and a valid ``FRONTEND_URL`` are always required. ``TRUST_PROXY_HOPS`` must be a non-negative integer string or the process aborts before binding a port.

3. Provisioning D1 and R2 (once, per environment)

1. Auth wrangler on the operator machine: ``npx wrangler login``.
2. D1 database — staging already exists and is the committed binding target in ``wrangler.jsonc`` (``database_name: "xolvon-staging"``, binding ``DB``, ``migrations_dir: "src/database/migrations"``). Production requires a new

- `npx wrangler d1 create xolvon-production` (name TBD by owner); record the resulting id ****only**** in the secret store as `CLOUDFLARE\_D1\_DATABASE\_ID` – never in documents.
- 3. R2 bucket: `npx wrangler r2 bucket create <bucket>`; mint an API token scoped to read/write on that bucket; store endpoint/keys as the four `R2\_\*` env names.
- 4. Deployment target for the Node process: ****not yet provisioned**** – any Node-capable host/PaaS that can hold a secret store and sit behind HTTPS with a known proxy hop count. Decide and record it in [`decision-log.md`](decision-log.md) before first deploy (relates to PRD §91.4, open).

4. Migration apply

- ****Staging/production (D1):****
`npx wrangler d1 migrations apply xolvon-staging --remote` (repeat with the production database name for production). Take a D1 backup before the first apply of any release.
- The in-repo runner (`src/database/migrate.ts`) applies files in `src/database/migrations/` in filename order, records each with a SHA-256 hash in `\_migrations`, skips applied files, and ****throws on hash drift**** – applied migrations are immutable. The HTTP server does not auto-migrate.
- Never edit an applied migration; add a new numbered file (see README.md §5).

5. Staging → production deployment procedure

Do staging first; production is the same sequence with production values.

5.1 Staging

1. Prereqs from §0 satisfied; backup of the staging D1 taken.
2. Apply pending migrations (§4) against the staging database.
3. Deploy the built artifact (`npm ci && npm run build` → ship `dist/`, or the provider's equivalent) with the staging env populated (§2).
`package.json` exposes only `deploy: "nest deploy"` as a placeholder – it targets no configured staging/production platform, so the actual deployment mechanism is the provider target from §3.4 (still unprovisioned).
4. Health check: `curl -i https://<staging-host>/api` → expect 200. Verify headers on the response: CSP present with the configured origins, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, no `X-Powered-By`;
****no HSTS on staging**** (production-only, by design).
5. Functional smoke (clean accounts, throwaway data):
 - `POST /api/auth/register` then `login`; repeat login 6× → 6th returns 429 (auth 5/min).
 - `GET /api/courses` → 200; burst 31× → 429 (catalog 30/min).
 - Browser-mutation integrity: `POST /api/orders` with a disallowed `Origin:` header → 403; without browser headers + valid Bearer → passes auth path.
 - CORS: allowed origin `OPTIONS` preflight returns `Access-Control-Allow-Origin` + credentials; unknown origin does not.
6. First admin (§6).
7. Record evidence (status codes, header dumps) without copying tokens or secret values anywhere.

5.2 Production

1. Same as staging with production secret store, plus:
2. `APP\_ENV=production` → HSTS `max-age=31536000; includeSubDomains` must appear; confirm only after HTTPS is terminating correctly, since HSTS is sticky in browsers.
3. `CORS\_ALLOWED\_ORIGINS` / `CSP\_CONNECT\_SRC` set to the final owner-approved domains (blocked on DL-027 – OPEN).
4. Repeat the §5.1 smoke matrix against the production host on a clean account, then promote real content.
5. Any production success claim requires captured live evidence – the current recorded state is ****BLOCKED**** ([`qa-results.md`](qa-results.md)).

6. First-admin bootstrap (controlled)

There is no public admin registration route and no role input is ever accepted from a client (DL-005/DL-017). Seed the first admin from a controlled operator shell with the target environment's secret store attached, using the variable names only (values from the secret store; avoid shell history – e.g. read them from a file descriptor or interactive prompt):

```
```bash
ADMIN_BOOTSTRAP_EMAIL='<admin email>' \
ADMIN_BOOTSTRAP_PASSWORD='<from secret store>' \
npm run seed:admin
```
```

Behavior (`src/database/seed-admin.ts` → `admin-bootstrap.ts`): applies pending migrations, hashes the password with Argon2id via `PasswordService`, inserts `role='admin', status='active'`, prints only `created` or `already\_exists`, is idempotent for the same admin email, and **refuses** (conflict error) to touch an existing non-admin account.

7. Rollback

Authoritative procedure: [`production-operations.md`](production-operations.md)
"Rollback procedure". Short form, in order:

1. Stop promotion; record deployment id, last-applied migration filename, and incident timestamp (no secret values).
2. **Revert the deploy** to the last known-good immutable build and confirm it boots with the target secret store (`GET /api` health).
3. **Revert the last migration** only if the failed release applied one: restore the pre-migration database backup/snapshot taken before that apply. The runner is forward-only and creates no down-migrations.
4. If the platform cannot restore, ship a **compensating migration** as a new numbered file that reverses the affected change. **Never** edit an applied migration – `\_migrations` hash drift must fail closed.
5. Re-run the migration set against a disposable local copy first; verify `PRAGMA foreign\_key\_check` and the affected flows; redeploy known-good.
6. Keep the failed release isolated until root cause + forward fix are reviewed.

8. Monitoring & alerting – OPEN DECISION

`MonitoringPort` (`src/observability/monitoring.port.ts`:
`captureError`, `recordEvent`) is **interface-only** – no provider SDK, endpoint, recipients, or credential is wired (DL-022). Log/monitor hygiene in the meantime (enforced by `AllExceptionsFilter` and code review): event names, route/status context, non-sensitive identifiers only; never passwords, tokens, hashes, signed URLs, credentials, or echoed untrusted input.

Before any production monitoring wiring, a new decision entry must record provider, alert recipients, data boundary, and secret-store location. Until then, treat "monitoring" as: structured stdout logs + provider platform metrics + the \$5.1/\$5.2 smoke checks per release.

9. Post-deploy checklist (per release)

- [] Migrations applied cleanly (count matches expected new files; no drift)
- [] Health `GET /api` 200 through HTTPS
- [] Header matrix correct for the env (HSTS only in production)
- [] CORS allow/deny spot check; no wildcard-with-credentials
- [] Auth 5/min + one named group 429 observed
- [] Disallowed-Origin browser mutation → 403
- [] Admin seed/verify/activate/revoke happy path on clean accounts
- [] Evidence captured with zero secret values
- [] `qa-results.md` / runbook updated with real status (or an honest BLOCKED)